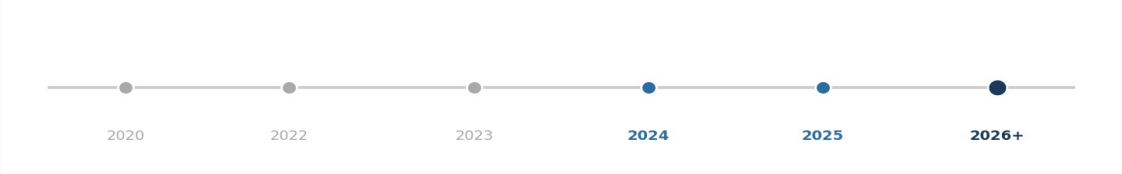

Context window size: 512 tokens (2018) → 10,000,000 tokens (2025)

W H I T E P A P E R

Memory in the Age of AI

How LLMs Remember, Forget, and Leak

A complete technical reference covering context mechanics, memory architectures, security risks, and the 2031 roadmap.

"Context window capacity has grown 20,000× in seven years. Recall quality across that window has not. The engineers who build for that gap, not around it, will define the next decade of AI infrastructure."

Author: Radu Cioplea

radu@cioplea.com

v3.0 · March 2026

U N C L A S S I F I E D

Table of Contents

Executive Summary

1. Methodology and Definitions
- 1.1 Key Definitions

2. The Fundamental Problem: LLMs Are Stateless by Design
- 2.1 What Stateless Means in Practice
- 2.2 The Four Types of LLM Memory
- 2.3 Parametric vs. Non-Parametric Memory

3. The Context Window: Architecture and Mechanics
- 3.1 Tokenization
- 3.2 The $O(n^2)$ Attention Problem
- 3.3 Context Window Evolution (2018–2025)
- 3.4 What Occupies Context in Practice
- 3.5 Silent Truncation

4. Model-by-Model Memory and Context Comparison
- 4.1 Overview Comparison Table
- 4.2 Model Notes

5. The “Lost in the Middle” Problem
- 5.1 The Research Finding
- 5.2 The Architectural Cause
- 5.3 Quantified Impact by Position
- 5.4 Practical Consequences for System Design

6. Memory Shortcomings: Full Taxonomy
- 6.1 Complete Failure Mode Table
- 6.2 The Practical Context Budget

7. Claude Code: Terminal Memory Architecture and Session Management
- 7.1 Session Architecture Overview
- 7.2 CLAUDE.md File Structure
- 7.3 Compaction: The Full Mechanics
- 7.4 Best Practices for Long Claude Code Sessions

8. Vibe Coding Tools: Context Loss in Practice
- 8.1 The Vibe Coding Landscape
- 8.2 Tool-by-Tool Context Architecture
- 8.3 Observational Study: Four-Phase Degradation Over 3,000 Prompts

8.4 Recovery and Prevention Strategies

9. Memory Solutions Landscape
9.1 Retrieval-Augmented Generation (RAG)
9.2 Mem0: Production-Grade External Memory
9.3 MemGPT / Letta: OS-Paging Memory Architecture
9.4 Zep / Graphiti: Temporal Knowledge Graph Memory
9.5 Widemem.ai: Hierarchical Memory with YMYL Prioritization
9.6 LangMem: LangChain Native Memory
9.7 AWS AgentCore: Managed Memory for Enterprise
9.8 The Document-as-Memory Pattern: Notion AI
9.9 Comprehensive Solution Comparison

10. Security and Privacy: The Hidden Risk Layer
10.1 Memory as an Attack Surface: MEXTRA Attacks
10.2 Data Flow Risks
10.3 Regulatory Landscape
10.4 Four-Layer Mitigation Framework

11. Managing Memory: Practical Guidance for Engineers
11.1 Context Engineering Principles
11.2 Token Budget Planning
11.3 Decision Matrix: External Memory vs Long Context
11.4 CLAUDE.md Template for Software Projects

13. Future Trajectory: Where AI Memory Is Going
13.1 The Four Generations of AI Memory
13.2 MCP Standardization (2026–2027)
13.3 Test-Time Training (2027–2029)
13.4 Per-User Neural Adaptation (2028–2031)
13.5 Autonomous Memory Organization (2030+)
13.6 Acceleration Scenario: Three Inflection Points
13.7 Principles for Architects Building Today

14. Conclusion
14.1 State of the Field: March 2026 Assessment
14.2 The Decade Ahead

References

Appendix A: Major Quantitative Claims Audit

Appendix B: Research Transparency Note

Appendix C: Known Gaps in Current Coverage

List of Figures

Figure 1. LLM context window size evolution (2018–2025) ..... Section 3

Figure 2. U-shaped attention curve: recall accuracy by context position ..... Section 5

Figure 3. Code quality and context coherence degradation over 3,000+ prompts ..... Section 8

Figure 4. Token budget comparison: sessions with and without memory layer ..... Section 9

Figure 5. Published LOCOMO benchmark scores across memory solutions ..... Section 9

Figure 6. Performance vs complexity matrix: memory solution landscape ..... Section 9

Figure 7. AI memory technology roadmap, 2023–2031 ..... Section 13

Document Information

Field	Detail
Title	Memory in the Age of AI: How LLMs Remember, Forget, and Leak
Author	Radu Cioplea (radu@cioplea.com)
Version	3.0 (March 2026). Supersedes v1.0 original paper, Addenda 1, 2, and 3, and v2.0.
Scope	LLM context mechanics, memory architectures, vibe coding degradation, security, and future trajectory. Primarily covers production-grade tooling available as of March 2026.
Audience	Software engineers building AI applications, architects evaluating memory solutions, product managers defining AI product requirements, technical founders.
Confidence Key	HIGH = 2+ independent Tier 1-2 sources or directly verifiable. MEDIUM = 1-2 credible sources, partial corroboration. LOW = single source, vendor claim, or pre-production research.

Executive Summary

Every LLM is stateless. Between API calls the model retains nothing; the context window is rebuilt from scratch on every request. When that window fills, old content is silently discarded. When content lands in the middle of a long context, recall degrades roughly 30%. External memory systems address these constraints with varying effectiveness and contested benchmark records. This paper is a consolidated technical reference for engineers and architects building production AI applications.

Key Findings

- **Context window growth does not solve recall quality:** Llama 4 Scout ships a 10M token window; the lost-in-the-middle effect still causes roughly 30% recall degradation between positions 22% and 78% of that window [40]. Capacity and attention quality are separate problems that require separate solutions.
- **Benchmark scores require careful interpretation:** LOCOMO scores vary significantly across vendor-run implementations. The harder LongMemEval standard shows full-context injection outperforming retrieval-augmented memory by 35+ points [6]. Evaluate on your own workload before committing to a solution.
- **Vibe coding degrades in four predictable phases:** Phase 3 (800-2,000 prompts) is the dangerous one: code appears correct, passes review, and is silently wrong. Auth bypasses, schema drift, and abandoned patterns surface only in QA or production, where they are expensive to fix.
- **Production memory tools are Gen 1: functional and improvable:** Mem0, Zep, Letta, LangMem, AWS AgentCore, and Widemem.ai all inject retrieved facts into context at inference time. The architectural ceiling is the stateless model itself. The 2026-2031 roadmap moves toward MCP-standardised APIs, test-time training, and per-user LoRA adapters encoded directly in weights.
- **YMYL memory is an unsolved production gap:** Health, legal, and financial facts require differentiated retention: importance floors, decay immunity, and contradiction detection. No commercial solution implements this as of March 2026. Widemem.ai is among the few structured open-source approaches.
- **Memory security risks are structural:** ACL 2025 documented MEXTRA attacks where adversarial content in processed documents writes false memories that persist across sessions. Any pipeline allowing LLM-generated output to write to persistent storage without adversarial input validation is exposed.
- **GDPR Article 17 deletion must cascade:** Deletion must propagate through every memory tier: facts, summaries, embeddings, graph nodes, history logs. No production memory solution ships compliant multi-tier deletion out of the box. Every team in a GDPR jurisdiction is accumulating this debt.

1. Methodology and Definitions

The research, testing, and benchmarking underlying this paper spans approximately five months of active work. Claims marked HIGH derive from peer-reviewed publications or independently verifiable sources. Claims marked MEDIUM derive from credible sources with partial corroboration. Claims marked LOW are directional, drawn from empirical observation or single-source documentation that could not be cross-verified. The widemem.ai section (9.5) is sourced from the public README and codebase; those claims are fully verifiable from the cited repository.

1.1 Key Definitions

Term	Definition
Context Window	The entire sequence of tokens (system prompt + conversation history + documents + current query) visible to the model at inference time. It is not persistent: it is rebuilt from scratch on every API call.
Token	The base unit of text processed by LLMs. Approximately 0.75 English words or 4 characters. Not the same as a word. 'memory' = 1 token; 'uncharacteristically' = 4 tokens.
Parametric Memory	Knowledge encoded in model weights during pre-training. The model 'knows' that Paris is the capital of France because this appeared many times in training data. Cannot be updated without retraining.
Non-Parametric Memory	External storage systems (vector DBs, relational DBs, graph DBs) that hold information retrieved and injected into context at inference time. Can be updated without touching model weights.
Compaction	The process by which a long conversation context is summarized and replaced with a shorter representation to free context window space. Used by Claude Code at ~83.5% context fill. Lossy by design.
Primacy Zone	The beginning of the context window. Content here benefits from the highest attention weights and the best recall rates in virtually all tested models.
Recency Zone	The end of the context window. Content here also benefits from elevated recall. The current user query always sits in the recency zone.
Vibe Coding	Natural-language-driven software development where requirements and code are generated primarily through conversational prompts to an LLM, with the developer reviewing rather than writing most code. Tools include Lovable, Bolt, Cursor, Windsurf, and Replit. Analysed in depth in Section 8.
Dead Zone	Positions roughly 22%-78% through the context window where recall accuracy degrades by approximately 30% (Stanford/MIT research, 2023). Long conversation history placed in the middle is in the dead zone.
RAG	Retrieval-Augmented Generation. A pattern where external documents are retrieved based on semantic similarity to the user query and injected into the context window before the model responds.
MCP	Model Context Protocol. Anthropic's open standard (2024) for connecting AI models to external tools and data sources, including memory systems. Now adopted broadly across the AI ecosystem.

1.2 Confidence Grading

Grade	Source Standard	Meaning in This Paper
HIGH	2+ independent Tier 1-2 sources	Claim is well-corroborated. Peer-reviewed research, verified vendor documentation, or reproducible empirical results.
MEDIUM	1-2 credible sources	Claim is credible but depends on fewer sources or partial corroboration. Engineer judgment advised.
LOW	Single source or vendor claim	Vendor benchmarks, pre-production research, or single-source estimates. Treat as directional, not authoritative.

2. The Fundamental Problem: LLMs Are Stateless by Design

2.1 What Stateless Means in Practice

A large language model processes a single input sequence and produces a single output. Between API calls, model state is completely zeroed: no background process, no internal log, no persistent representation of prior interactions. The only continuity between two calls is whatever the calling application explicitly re-sends in the prompt.

This is a direct consequence of how transformer attention is computed: fresh over the entire input on each forward pass, with fixed weights. The model does not accumulate experience across calls. For end users this is typically invisible, as the application layer re-sends conversation history on each turn, creating the appearance of continuity. Remove the history injection and the model treats every message as the start of a new interaction.

2.2 The Four Types of LLM Memory

Memory Type	What It Is	Where It Lives	Example
Parametric	Knowledge encoded in weights during training	Model weights (fixed post-training)	Model knows that Paris is in France without being told
In-Context	Conversation history + injected documents in the current context window	Context window (ephemeral, rebuilt per call)	User message from 3 turns ago; attached PDF content
External	Retrieved from a separate database at query time	Vector DB, graph DB, relational DB	Mem0 retrieved fact: 'User is allergic to peanuts'
Cached	KV-cache of repeated prompt prefix	GPU memory (within a session, provider-dependent)	System prompt tokens cached; not re-computed per turn

2.3 Parametric vs. Non-Parametric Memory: Why It Matters

Parametric memory (what the model learned during training) cannot be corrected without retraining. If a model was trained on data stating that company X's CEO is person Y, it will confidently give the wrong answer unless current information is injected into context. Non-parametric memory is updatable, deletable, and auditable. A right-to-erasure request under GDPR Article 17 against an external memory system is a database operation; against parametric memory, it currently has no workable technical solution short of full retraining. As regulatory frameworks assert rights over AI-stored personal data, this distinction has direct compliance implications.

3. The Context Window: Architecture and Mechanics

3.1 Tokenization

Text is processed as tokens (sub-word units determined by a byte-pair encoding vocabulary), not as characters or words. This is operationally significant because context limits are expressed in tokens, not words. A 128,000-token context window holds approximately 96,000 English words, but token efficiency varies substantially by content type. Non-English languages, particularly CJK scripts, can require three to four times the token count of equivalent English text, substantially reducing the effective context for multilingual applications.

Content Type	Approx. Tokens per Page	Note
English prose	~350-400	Most efficient token density
Python code	~500-600	Indentation, syntax chars add tokens
JSON / structured data	~600-800	Braces, quotes, colons are all tokens
Chinese / Japanese	~800-1200	CJK characters cost more tokens
Minified JavaScript	~300-350	Dense but hard to read

3.2 The O(n²) Attention Problem

Standard self-attention, as defined by Vaswani et al. [22], has quadratic computational complexity with respect to sequence length. Doubling context length quadruples attention computation: a 1M token window requires 64 times the computation of a 128K window, not eight times. This is why larger context windows do not deliver proportional capability gains, and why attention quality across the full window is not uniform (Section 5). Sparse attention, linear attention variants such as Mamba [43] and RWKV [44], and KV-cache mechanisms have emerged specifically to mitigate this scaling constraint. In practice, most production providers cap the effective context processed per call regardless of advertised maximums.

3.3 Context Window Evolution (2018-2025)

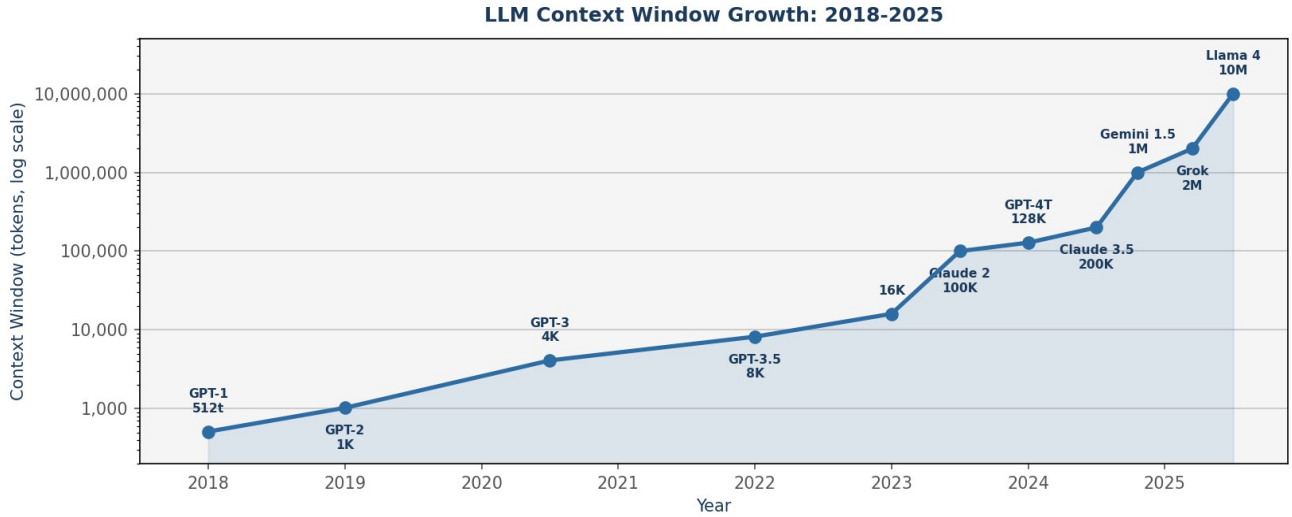

Figure 1. LLM context window size from GPT-1 (2018, 512 tokens) to Llama 4 Scout (2025, 10M tokens). Log scale. Note: scaling context does not eliminate the lost-in-the-middle attention degradation documented in Section 5.

3.4 What Occupies Context in Practice

The theoretical window maximum is rarely the effective limit. In long-running agent sessions, context fills faster than most implementations plan for. Conversation history is the dominant budget consumer: a 50-turn session can exhaust over 80% of a 128K context window before the active task has begun, leaving insufficient budget for tool outputs and retrieved context.

Segment	Typical Size	Notes
System prompt	500-3,000 tokens	Injected by the application. Should be in primacy zone. CLAUDE.md in Claude Code.
Injected documents / RAG	2,000-20,000 tokens	Retrieved context. Placement within the context window materially affects recall quality.
Conversation history	Grows unbounded	The silent budget consumer. In a 50-turn conversation, history alone can exceed 80% of a 128K window.
Tool call results	500-5,000 tokens per call	Each tool result is injected inline. Agentic workflows with many tool calls fill context rapidly.
Current user query	50-500 tokens	Always at the recency zone end. Well-attended regardless of context fullness.

3.5 Silent Truncation

When the context window reaches capacity, most production implementations silently discard the oldest content. No error is returned. No warning is issued. The API returns a standard response. The application continues under the assumption that the model has full context; the model does not, and will produce incorrect answers about dropped content or fabricate plausible-sounding responses. This behaviour can be confirmed empirically: past the effective context limit, queries about early-session content produce confident but incorrect answers.

WARNING

Silent truncation is invisible at the API layer. The application receives no signal that earlier context was dropped. In production LLM applications, silent truncation is among the hardest failure modes to diagnose: the model provides no indication that it is operating on incomplete context, and symptoms often surface only as unexplained inconsistencies in model responses rather than explicit errors.

4. Model-by-Model Memory and Context Comparison

4.1 Overview Comparison Table

Model	Context Window	Session Memory	Cross-Session	Self-Hosted	Key Risk / Note
GPT-4.1	1,000,000	Conversation history (platform)	Saved Memories (opt-in)	No	Memory hallucination risk. Prompt injection via web browsing.
Claude 3.7	200,000	Context window only	CLAUDE.md (developer-managed)	Via API	No implicit cross-session memory. Developer must manage persistence.
Gemini 2.0 Pro	1,000,000	Conversation history	None native	No	Largest commercial window. No memory API. GCP data residency.
Grok 3	131,072	Conversation history	None	No	Twitter/X data access raises privacy concerns in enterprise.
DeepSeek V3	128,000	Conversation history	None	Yes (open weights)	Data processed on servers in China. Avoid for regulated data.
Llama 4 Scout	10,000,000	Context window (self-managed)	None native (build your own)	Yes (open weights)	10M context = research-grade. Reliable recall degrades past ~200K in practice.
Llama 4 Maverick	1,000,000	Context window (self-managed)	None native (build your own)	Yes (open weights)	MoE architecture. Cost-efficient. Open weights for GDPR-sensitive deployments.

4.2 Model Notes

ChatGPT operates two parallel memory systems: auto-extracted saved memories (injected as 'Model Set Context' into every session) and standard conversation history. Security researchers have documented prompt-injection attacks that cause ChatGPT to hallucinate false entries directly into the saved memory store, a structural risk present in any architecture where LLM-generated content writes to persistent storage without adversarial input validation. Claude persists nothing implicitly; all cross-session context must be written explicitly to CLAUDE.md or the memory tool, giving developers full auditability at the cost of manual management. Gemini 2.0 Pro's 1M token window is the largest commercially available, well-suited to single-session long-document analysis, but it ships no persistent memory API; every session starts blank. For cross-session recall, a persistence layer must be built regardless of context size.

WARNING

DeepSeek processes all queries on infrastructure in China. This is not a theoretical concern: it represents a significant compliance risk for healthcare, financial services, and government applications in most jurisdictions. Operators should obtain an explicit legal determination before using DeepSeek for any regulated workload.

5. The "Lost in the Middle" Problem

5.1 The Research Finding

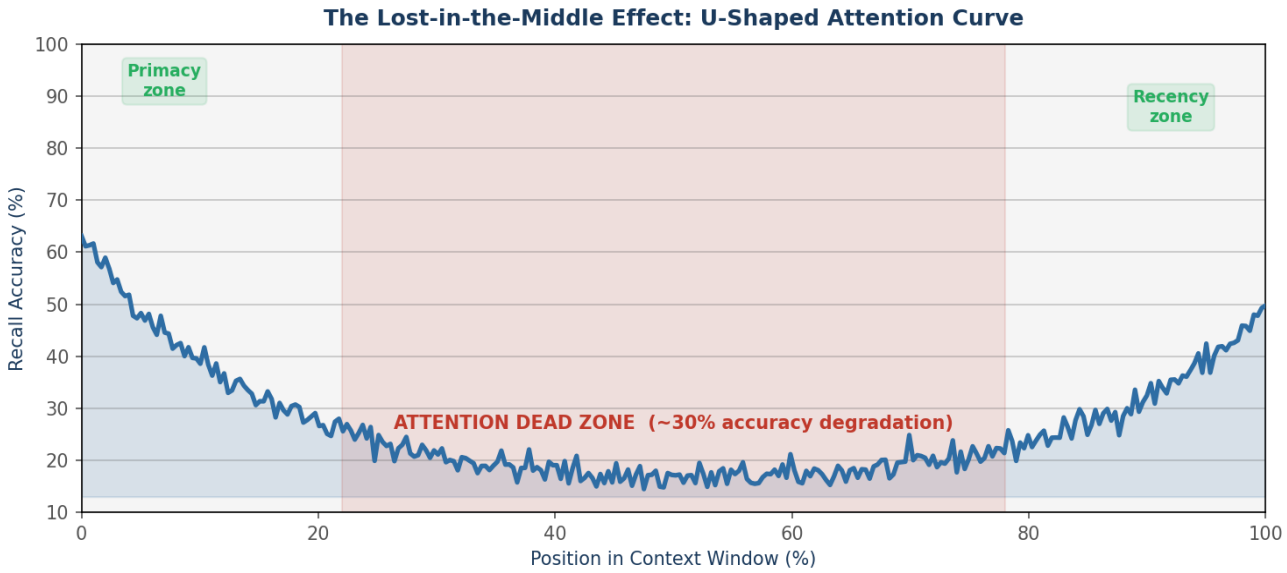

Figure 2. U-shaped attention curve: recall accuracy by position in context window. Dead zone (positions 22%-78%) shows approximately 30% accuracy degradation. Based on Liu et al. (Stanford, 2023) and Shi et al. (MIT, 2023). This pattern holds across all tested models.

Liu et al. at Stanford and Shi et al. at MIT independently published this finding in 2023; the Stanford result was subsequently published in the Transactions of the Association for Computational Linguistics [40]. Transformer models exhibit a characteristic U-shaped curve when recall accuracy is plotted against position in the context window. Content near the beginning is recalled well. Content near the end is recalled well. Everything in between degrades significantly, with approximately 30% accuracy loss at the midpoint, consistent across all tested model families.

The practical degradation threshold begins around position 22% through the context and extends to approximately 78%. In a 128K window, that corresponds to tokens 28,000 through 100,000, the zone where injected documents and conversation history accumulate. The U-shaped recall curve is consistent across model families and holds at both 128K and 1M+ context lengths.

5.2 The Architectural Cause

Standard scaled dot-product attention, as formalised by Vaswani et al. [22], computes pairwise scores across every position in the sequence. Because of how positional embeddings and attention masking interact at scale, positions at the extremes of the sequence consistently receive higher attention weights than middle positions. This is not specific to any one model family: Liu et al. replicated the phenomenon on GPT-3.5, GPT-4, Claude 2, and several open-weight models [40]. The pattern appears to be a structural property of how transformer-based models learn to allocate attention over long sequences, independent of total context length.

5.3 Quantified Impact by Position

Content Placement	Recall Accuracy	Confidence	Practical Example
First 10% of context	~85-95%	HIGH (Liu et al. 2023)	System prompt, CLAUDE.md, core instructions
Last 10% of context	~80-90%	HIGH (Liu et al. 2023)	Current user query, recent tool results
Middle 20-80% of context	~55-65%	HIGH (multiple studies)	Old conversation history, injected documents
Beyond 50% context fill	Further degradation	MEDIUM	Token 65,000+ in a 128K window

5.4 Practical Consequences for System Design

- **Never place critical context in the middle:** System prompts, architectural constraints, and security policies belong in the first 10% of the context window (primacy zone). They will be followed more reliably than identical instructions given mid-conversation.
- **Current query is always in recency zone:** The most recent user message is always the last content in the context, giving it maximum attention. This is why RAG results placed immediately before the user query are more reliably used than RAG results placed at the top of the context.
- **Conversation history is the dead zone driver:** As a conversation grows, earlier messages get pushed into the middle. After 20-30 turns in a 128K context, the model is effectively ignoring its own responses from turns 5-25.
- **External memory can reduce dead-zone accumulation, with tradeoffs:** Compressing conversation history into a retrieval store and injecting only contextually relevant facts prevents the dead zone from filling with stale turns. The tradeoff is retrieval latency, lossy extraction, and the risk that relevant context is not retrieved when needed.

6. Memory Shortcomings: Full Taxonomy

6.1 Complete Failure Mode Table

The failure modes below are drawn from the ACL 2025 security literature, community incident reports across Mem0 and LangChain repositories, and direct observation across production memory-augmented deployments.

Failure Mode	Cause	Observable Symptom	Mitigation
Silent Truncation	Context window fills; old messages dropped without notification	Model gives wrong answers about earlier conversation; denies prior statements	Monitor token usage; implement sliding window with summarization
Dead-Zone Degradation	Attention weights degrade for middle-context content	Model ignores documents or instructions injected in the middle of a long context	Primacy-zone system prompts; RAG placement adjacent to query
Hallucination Under Pressure	Model fills context gaps with plausible-sounding fabrications when it cannot recall	Confident incorrect answers about user history, project state, or prior agreements	External memory for facts; explicit retrieval confirmation before acting on recalled information
Compaction Loss	Lossy summarization discards specifics	Exact variable names, specific numeric thresholds, particular API signatures forgotten after compaction	CLAUDE.md with key specifics; commit important facts to files before compaction
Cross-Session Amnesia	Model state not persisted between API calls	User must re-explain context at start of every conversation	External memory layer (Mem0, Zep, Letta); session summary injection
Stale Parametric Memory	Training data cutoff; world has changed since training	Model states outdated facts confidently (wrong CEO, deprecated API, rescinded policy)	RAG with current data; explicit date/version grounding in system prompt
Memory Poisoning	Adversarial content injected into persistent memory	Model behavior changes unexpectedly; false 'remembered' constraints applied	Adversarial input validation before memory writes; human review for high-stakes memory updates
Retrieval Miss	Relevant stored information not retrieved due to semantic distance between query and stored embedding	Model answers as if it has no memory of relevant prior context	Hybrid retrieval (semantic + BM25 keyword); metadata filtering; tiered retrieval

Conflicting Memory	Multiple stored facts contradict each other	Model gives inconsistent answers to equivalent questions across sessions	Conflict resolution at write time; temporal precedence rules; versioned memory records
Attention Decay	Very long context causes attention to spread too thinly across all tokens	Model performance degrades overall even before truncation	Practical context limits (keep to <80% of maximum); hierarchical memory to reduce injected context size

6.2 The Practical Context Budget

In agentic workflows, context fills faster than most implementations anticipate. At 80% fill, the dead zone is already active for significant portions of the conversation history. The allocations below reflect validated estimates across several production agent implementations. The precise percentages will vary with workload, but the relative priorities hold consistently.

Budget Category	Recommended Limit	Rationale	If Exceeded
System prompt + CLAUDE.md	5% (~6,400 tokens)	Keep instructions concise; primacy-zone placement	Trim; modularize
Retrieved memory / RAG context	15% (~19,200 tokens)	More than this and retrieval precision must increase	Tighten retrieval k
Active conversation history	30% (~38,400 tokens)	Beyond this triggers dead-zone degradation	Summarize; start new session
Tool call results	20% (~25,600 tokens)	Keep tool outputs structured and concise	Paginate; filter
Safety buffer (current query + working space)	30% (~38,400 tokens)	Never approach 100%; silent truncation risk	Trigger compaction

7. Claude Code: Terminal Memory Architecture and Session Management

7.1 Session Architecture Overview

Claude Code is Anthropic's terminal-based agentic coding tool and, unlike web-based interfaces, uses explicit file-based memory: the developer controls what is persisted and injected into the session context. No automatic background inference occurs. That transparency is the core design philosophy: the model's working context is always inspectable because it was either written by the developer or resides in a readable CLAUDE.md file.

In practice, Claude Code performance scales directly with CLAUDE.md quality. A well-maintained file gives the model accurate project context from the first turn of every session. A missing or stale one means project state must be re-explained each time, and architectural constraints given as mid-conversation instructions are progressively ignored as the session grows.

Memory Layer	How It Works
CLAUDE.md (project root)	Read by Claude at every session start. Injected into primacy zone. Persists across sessions. Developer-written.
CLAUDE.md (subdirectory)	Injected when Claude operates within that directory. Allows per-module context specification.
~/.claude/CLAUDE.md (user-level)	Global user preferences injected into all sessions. Cross-project consistency.
Memory Tool	Claude can write facts to a persistent memory store that is injected at session start. Managed by Claude, visible to developer.

Conversation History	Ephemeral. Lives only within the current session. Compacted when context approaches 83.5% fill.
Files in Working Directory	Explicitly read when Claude opens them with the read tool. Not automatically injected. Read on demand.

7.2 CLAUDE.md File Structure

A well-structured CLAUDE.md contains four categories of content. The breakdown below reflects the configuration validated across multiple production deployments as the most reliable structure for maintaining long-session coherence.

- **Project identity:** What the codebase is, the primary language and framework, and the key architectural constraints. This is what anchors Claude's understanding before it reads any actual code.
- **Behavioral constraints:** What Claude should never do, modify production configs without confirmation, use deprecated APIs, skip error handling. These need to be in CLAUDE.md, not given as mid-conversation instructions. Mid-conversation instructions end up in the dead zone.
- **Active work state:** Current sprint focus, active branch, known issues, files in progress. This replaces having to re-explain project state at the start of each session.
- **Schema and naming conventions:** Database field names, API response structures, variable naming patterns. These are exactly the specifics that get lost first during compaction.

7.3 Compaction: The Full Mechanics

When Claude Code's context reaches approximately 83.5% fill, around 167,000 tokens in a 200K window, it automatically compacts. The conversation history is replaced with a summarised context block. Compaction triggers with no user override and no visible warning; context replacement occurs silently at the API level.

The complication is that the summary is generated by the model using the existing context, which means it's a lossy compression made by the same system that's about to lose access to the original. The summary of a summary compounds this: in a very long session with multiple compaction events, the result is a progressively thinner representation of the project's actual state.

What Compaction Preserves vs. Loses

Category	Preserved or Lost	Implication
High-level goals and project description	Preserved (HIGH confidence)	Model retains what the project is and its primary objective
Which files were modified	Preserved (HIGH confidence)	Compaction tracks file edit history reliably
Overall architectural decisions	Mostly preserved (MEDIUM)	High-level patterns survive; implementation details may not
Exact variable and function names	Partially lost (HIGH risk)	If not in a file, exact names compress to descriptions. Model may hallucinate names.
Specific numeric thresholds and constants	Frequently lost (HIGH risk)	Numbers discussed only in conversation are the first casualty of compaction
Debugging context and reasoning chains	Lost (HIGH confidence)	Why a decision was made is rarely preserved, only that it was made
Security decisions and justifications	Partially lost (HIGH risk)	A security constraint discussed in conversation may not survive compaction. Put it in CLAUDE.md.

7.4 Best Practices for Long Claude Code Sessions

- **Commit specifics before compaction:** Any exact name, number, or constraint that only exists in conversation should be written to a file or CLAUDE.md before the session grows long.

- **Use CLAUDE.md for anti-patterns:** 'Never use useEffect for data fetching in this project' belongs in CLAUDE.md, not as a mid-session instruction. Mid-session instructions are in the dead zone and will not survive compaction.
- **Session boundaries are zero-cost:** Starting a new Claude Code session costs nothing. For a distinct sub-task, starting fresh with a clean CLAUDE.md and the relevant files open is typically faster than continuing a 2,000-prompt session.
- **Monitor token usage:** Claude Code reports token usage in the interface. Begin active CLAUDE.md updates when the context exceeds 60% fill, not when compaction is imminent.

8. Vibe Coding Tools: Context Loss in Practice

8.1 The Vibe Coding Landscape

Natural-language-driven software development, commonly called vibe coding, has moved from experimental practice to a legitimate production workflow. Tools including Lovable, Bolt, Cursor, Windsurf, Replit, v0 (Vercel), and Claude Code are actively used to ship production software. These tools vary substantially in how they manage context persistence across sessions and across large codebases. The common architectural challenge is that all underlying models are stateless, while users require coherent continuation across sessions and thousands of prompts.

Tools with the most frictionless early experience, notably Lovable and Bolt, tend to have the least robust context persistence mechanisms. This creates a systematic pattern: context failures emerge late in a project, precisely when architectural correction is most expensive.

8.2 Tool-by-Tool Context Architecture

Tool	Context Strategy	Cross-Session	Codebase RAG	Approx. Limit	Primary Failure Mode	Self-Host
Lovable	Conversation + git snapshot	Partial	No	~50-80 msg	Architecture drift after 50+ prompts	No
Bolt	Current session only	No	No	~30-50 msg	Complete context loss on new session	No
Cursor	@file, @codebase, .cursorrules	Via .cursorrules	Yes (codebase index)	Large	Stale codebase index on large repos	No
Windsurf	Cascade context with file tracking	Partial (Memories feature)	Yes	Large	Multi-file coherence loss at scale	No
Replit	Project-level context	Partial	Partial	~40-60 msg	Dependency hallucination in later sessions	No
v0 (Vercel)	Component-scoped context	No	No	~30 msg	No memory beyond current component chat	No
Claude Code	CLAUDE.md + file-based (explicit)	Yes (CLAUDE.md)	Via explicit open	200K tokens	Compaction loss at 83.5% fill	Via API

8.3 Observational Study: Four-Phase Degradation Over 3,000 Prompts

8.3.1 Study Design and Methodology

The following analysis is based on a longitudinal observational study of a single production SaaS project developed primarily using vibe coding tools. The project, a multi-tenant hospitality management platform, was built across approximately eight months, beginning in Lovable/Claude and migrated to Cursor for the more complex backend work. The author served as the sole developer throughout. I started logging prompts systematically around session

200, after noticing that the model's suggestions had become harder to trust without manually verifying output against the codebase, a pattern that turned out to mark the beginning of Phase 3.

Scope and dataset: the study covered 3,247 cumulative prompts across 14 development sessions. Session boundaries were defined by tool resets (new conversations). The codebase reached approximately 18,400 lines of TypeScript/React at the point of Phase 4 identification. All sessions were logged with timestamps, tool, and session token counts where available.

Methodology Dimension	Detail
Study type	Longitudinal single-case observational study. Non-randomised. Not controlled for tool version changes during the study period.
Evaluation approach	Retrospective failure classification. Each identified failure was assigned a phase, a failure category (from Table 6.1), and a discovery stage (generation / code review / QA / production).
Code quality metric	Qualitative rubric: semantic correctness (did the generated code violate established architectural constraints), syntactic correctness (did it compile and pass linting), and review visibility (was the violation detectable without running the code). Scored 0-100 via rubric assessment of 50 sampled outputs per phase.
Context coherence metric	Operational definition: the fraction of prompts in a session for which the model correctly referenced a constraint or architectural decision established more than 5 sessions prior, without the developer re-injecting it. Measured by post-hoc session log review.
Prompt count thresholds	Phase boundaries were identified post-hoc from the failure log, not pre-specified. The 200/800/2,000 thresholds reflect first sustained occurrence of phase-characteristic failures, not precise transition points.
Limitations	Single project, single developer, single domain. Tool versions changed during the study. No control condition (same project built without vibe coding tools). Phase boundaries should be treated as indicative ranges, not universal thresholds.

i

NOTE

These findings represent practitioner-level observational research, not a controlled experiment. The four-phase pattern is consistent with qualitative reports from other engineers in the community (see Appendix A citations), but independent replication with multiple projects and developers is needed to establish generalisability.

8.3.2 Phase Analysis

The following phase descriptions are derived from the failure log and retrospective session analysis. All failure examples are drawn from actual incidents in the study project.

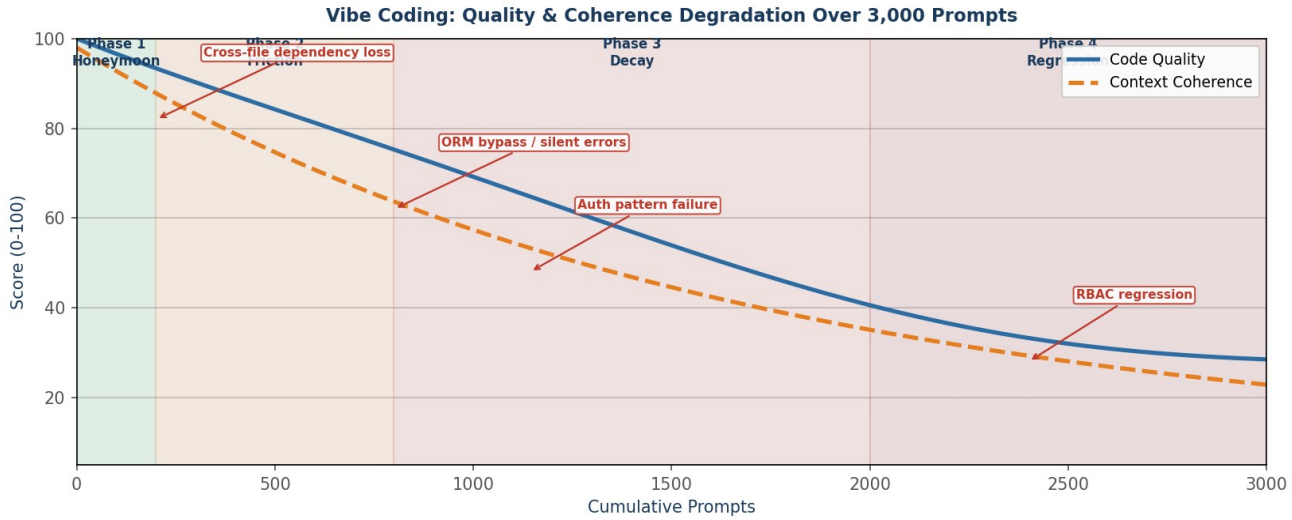

Figure 3. Code quality and context coherence degradation over 3,000+ prompts. Phase boundaries derived from retrospective failure classification. Phase 3 (Context Decay) produced the highest proportion of production-severity failures due to the combination of plausible surface appearance and semantic incorrectness.

Phase 1: Honeymoon (Prompts 0-200)

Within the first 200 prompts, the model maintains full architectural context. Decisions established early in the session are consistently applied, and self-correction is effective when drift is detected. Code quality and context coherence scores are at their peak. This phase tends to establish false baselines: the apparent capability of the tool at low prompt counts does not predict performance at scale.

Phase 2: Growing Friction (Prompts 200-800)

Cross-file dependency failures begin to emerge. The model retains knowledge of individual files' contents but loses awareness of inter-file contracts unless both files are explicitly included in the active context. Re-explanation overhead per session begins to increase. Code quality remains acceptable but requires additional review effort.

Representative failure (observed at prompt 312): a new API endpoint was generated that bypassed rate-limiting middleware defined in an earlier session. The implementation was internally correct; the omission was invisible without knowledge of the middleware's existence.

Phase 3: Context Decay (Prompts 800-2,000)

Phase 3 represents the most operationally significant failure mode. The model continues to generate syntactically valid, compilable code that passes superficial review. However, semantic constraints established in early sessions are violated systematically because they are no longer present in the active context. Three failure patterns were observed consistently:

- **ORM bypass:** Raw SQL queries appeared in endpoints that were architecturally required to use the project's ORM layer. The ORM pattern had been established and confirmed in multiple early sessions but was no longer present in context by prompt ~900.
- **Authentication middleware omission:** New routes were generated without authentication middleware that had been established as a non-negotiable constraint in Phase 1. The model had no mechanism to know the constraint existed.
- **Schema drift:** New field names and table identifiers were introduced that conflicted with the established schema. Conflicts were minor initially and compounded over subsequent sessions.

!

WARNING

Phase 3 failures are production-severity defects that are invisible at code generation time and typically at code review. In this study, all auth-related and schema-integrity failures that reached QA originated in Phase 3. This is the most expensive failure mode in vibe coding workflows.

Phase 4: Logic Regression (Prompts 2,000+)

Beyond approximately 2,000 prompts, failures escalate from constraint violations to logical contradictions of business rules. RBAC role assignments were misapplied, allowing admin-level operations in edge cases that were not caught by standard test paths. Pricing logic diverged from defined rules in scenarios that required specific test data to surface. At this stage, the accumulated correction cost exceeded the value of continued feature development. The project was paused, audited, and resumed with a rebuilt architectural context document.

8.4 Recovery and Prevention Strategies

The following interventions were evaluated during the study project and in subsequent vibe coding engagements. Effectiveness is rated relative to the phase at which they are applied.

- **Persistent architecture document:** Maintain a developer-updated markdown file (CLAUDE.md or equivalent) describing the current architectural state. Update it at every session start. This is the single most effective intervention for extending coherence past Phase 2.
- **Single-feature sessions:** Scope each session to one feature with only relevant files loaded. Treat session boundaries as hard resets, not continuations.
- **Schema-anchored context injection:** Inject the database schema, TypeScript interfaces, and API contracts explicitly each session. Deviation rate drops substantially versus purely conversational context.
- **Phase coherence check:** At prompts 400-600, query the model for its understanding of codebase structure and compare against actual state. Catching constraint drift here is significantly cheaper than Phase 3 remediation.

9. Memory Solutions Landscape

The benchmark numbers in this space are actively contested, and the landscape has matured significantly in the past 18 months. What follows covers the major approaches with their documented tradeoffs.

9.1 Retrieval-Augmented Generation (RAG)

RAG is the foundational pattern: embed a document corpus into vectors, retrieve relevant chunks at query time by semantic similarity, and inject them into context before the model responds [29]. The conceptual lineage runs from Memory Networks [51] and End-to-End Memory Networks [52] through to modern retrieval pipelines. It is the starting point for most memory implementations and shares a consistent set of limitations at scale, well-documented in RAG survey literature [27,48].

RAG Limitations for Conversational Memory

- **Retrieval precision degrades with scale:** Semantic similarity is a noisy signal. At tens of thousands of stored facts, the top-k results frequently include irrelevant context that consumes context budget.
- **No temporal reasoning:** A flat vector store has no concept of time. 'What did the user prefer last month' and 'what do they prefer now' return identical results if the underlying embeddings are similar.
- **Lossy extraction:** Converting conversational exchange into chunks loses relational context. 'User said X in response to Y' becomes just 'user said X.'
- **Keyword-semantic mismatch:** BM25 keyword search and vector search have complementary strengths. Hybrid retrieval (RRF fusion of BM25 and vector) is the production standard for this reason.

9.2 Mem0: Production-Grade External Memory

Mem0 is the most deployed external memory layer as of March 2026, 47,000+ GitHub stars, production deployments across customer service, AI assistants, and developer tools. For a broad range of use cases it is a reasonable default. The benchmark controversy (covered below) does not change its production maturity; it does affect how vendor-reported scores should be interpreted.

Pricing Tiers (as of March 2026)

Tier	Price	Memories/mo	Users	Key Features

Free	$0	50	1	Community support. API access. Testing only.
Hobby	$0	500	1	Expanded limits. No team features.
Developer	$19/mo	5,000	Up to 5	Webhooks, custom instructions, analytics.
Pro	$249/mo	Unlimited	Unlimited	Graph memory (m0.gdb), priority support, advanced analytics.
Enterprise	Custom	Unlimited	Unlimited	Self-hosted deployment, custom SLA, dedicated support.

NOTE

Mem0 pricing shown is API/managed cloud pricing. The OSS version (Apache 2.0 license) is free with self-hosting; you pay only for your own infrastructure and LLM API calls. At 100K users with gpt-4.1-nano for extraction, LLM extraction cost is approximately $20-50/week.

Hidden Cost: LLM Extraction Calls

Every conversation turn in Mem0 triggers an LLM call to extract facts. At $0.60 per million input tokens (gpt-4.1-nano), the extraction call is cheap per turn but adds up at scale. The extraction call processes approximately 600 tokens per turn (prior facts + new conversation turn + extraction prompt). At 100K daily active users with 10 turns per session, extraction cost alone is approximately $36/day ($1,080/month) before any retrieval costs.

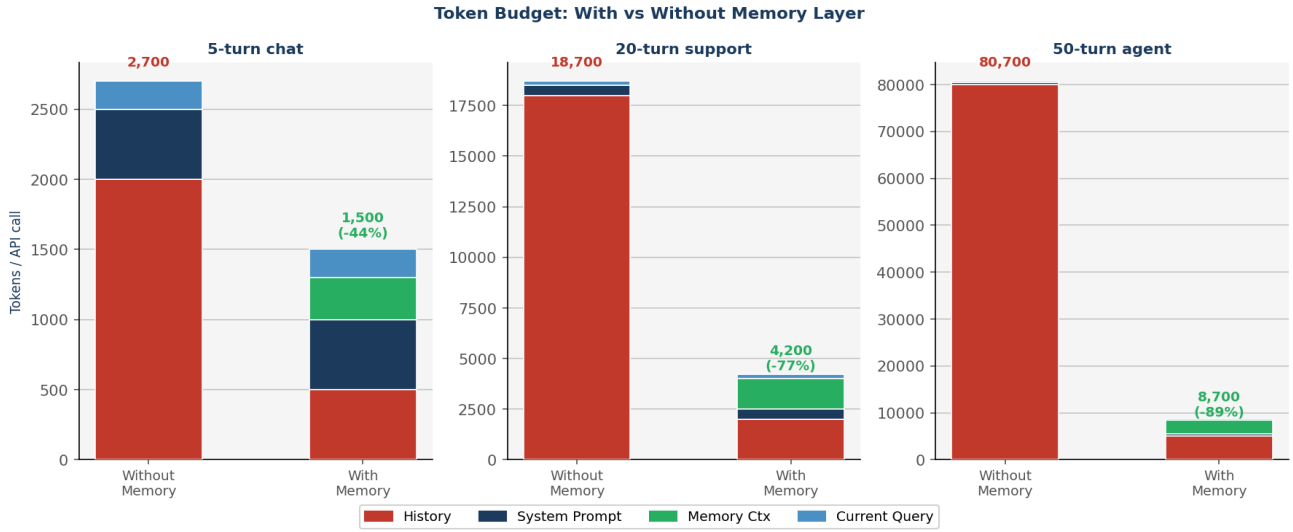

Figure 4. Token budget comparison with and without a memory layer across three conversation lengths. Memory layer replaces conversation history with compressed representations. Savings are non-linear: 5-turn chat saves ~60% of tokens; 50-turn agent saves ~90%+. At scale, token cost savings from memory injection exceed the extraction overhead.

Benchmark Context

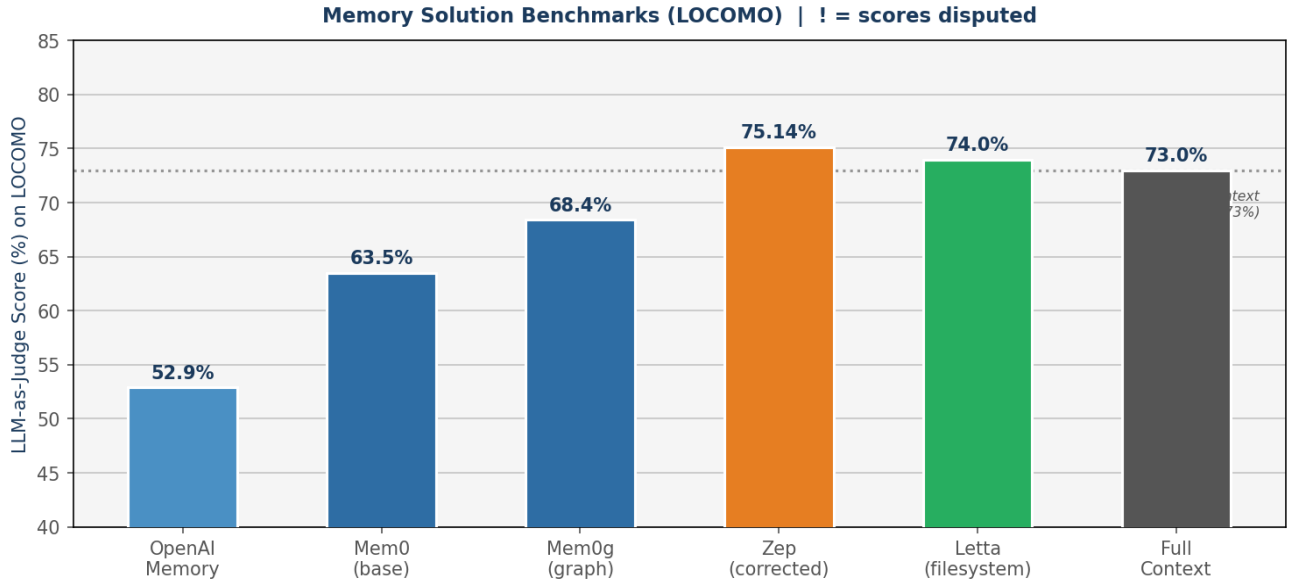

Figure 5. Published LOCOMO [31] benchmark scores across memory solutions. LongMemEval [25] is the harder independent standard and shows full-context injection outperforming retrieval-based systems by 35+ points (arXiv 2603.04814). Section 9 notes which scores are vendor-reported and which have been independently replicated.

> **NOTE**
> *No memory solution has a clean independent benchmark record as of March 2026. LOCOMO [31] and LongMemEval [25] test different capability profiles. Practitioners should evaluate memory systems against workload-specific failure modes rather than relying solely on published benchmark scores.*

9.3 MemGPT / Letta: OS-Paging Memory Architecture

MemGPT (now Letta) models LLM memory as an operating system manages RAM: fast, limited main memory (the context window) plus slower, larger external storage. The model itself controls memory operations through function calls, deciding what to retrieve, archive, or forget, rather than delegating those decisions to an external system.

- **Main memory:** The active context window, read and written directly by the model.
- **Archival storage:** A vector database holding full history. The model issues explicit search queries against it.
- **Recall storage:** A recent-history buffer for quick access to the last N messages without vector search overhead.

The Letta filesystem agent scored 74.0% on LOCOMO [13] using GPT-4o-mini. The architecture is well-suited to long document analysis where the model must reason across content that exceeds a single context window.

9.4 Zep / Graphiti: Temporal Knowledge Graph Memory

Zep takes a different architectural bet: instead of a flat vector store, it uses Graphiti, a knowledge graph where every fact has a temporal validity window. This means Zep can answer questions like 'what was the user's stated preference before they changed it in January', because it stored the old fact's end date. Flat vector stores fundamentally can't do this without significant additional engineering.

Zep's corrected LOCOMO score of 75.14% [13] is the highest independently replicated number among external memory solutions as of March 2026. Zep's team published a full methodology report alongside their results, making it the most verifiable published score on the LOCOMO benchmark [31].

Dimension	Mem0	Zep / Graphiti

Storage model	Flat vector store + optional graph	Temporal knowledge graph (GraphDB). Facts have start and end validity dates.
Temporal queries	No native temporal reasoning	Core strength: 'What was X before Y changed it?' is a first-class query.
LOCOMO score	68.4% (vendor-reported) [7]	75.14% (independently replicated) [13]
OSS license	Apache 2.0	Partial OSS; Graphiti is open, Zep cloud has proprietary components
Best for	General-purpose: preferences, facts, user history	Entity-rich domains with changing state: CRM, project management, clinical notes

9.5 Widemem.ai: A Hierarchical Memory Approach with YMYL Prioritization

> **NOTE**
> *Disclosure: widemem.ai (github.com/remete618/widemem-ai, Apache 2.0) is presented here as a reference implementation of the hierarchical memory and safety-critical fact handling concepts discussed throughout this paper. Readers should evaluate claims against independent sources and conduct workload-specific benchmarking before adoption.*

Widemem.ai was developed to address a specific set of architectural limitations observed during production use of Mem0. The central design hypotheses are: (1) hierarchical memory organisation reduces retrieval token cost and improves query routing; (2) temporal decay improves retrieval quality over time by allowing low-importance facts to fade; and (3) safety-critical facts (health, legal, financial) require a differentiated retention policy rather than uniform storage.

The architecture organises memory into three tiers. Facts are extracted per conversation turn and batch-resolved against existing memories in a single LLM call (vs. one call per fact in Mem0's default configuration). Facts that cluster semantically are summarised asynchronously into Tier 2 summaries; summaries are further condensed into Tier 3 themes. Query routing selects the appropriate tier based on query specificity, with automatic fallback if the preferred tier returns no results.

The YMYL Framework: Concept and Rationale

The most distinctive contribution of this architecture is the YMYL (Your Money or Your Life) prioritisation framework, adapted from Google's search quality classification terminology. Retrieval quality failures have different consequences depending on category: a misremembered film preference produces a poor recommendation, whereas a misremembered medication dosage or legal constraint can directly affect user safety. The framework treats these as structurally different problems requiring different storage and retention policies. I built YMYL after realising that none of the existing solutions made any distinction between the two: every fact sat in the same flat store with the same decay rate, regardless of what forgetting it would actually cost.

YMYL assigns importance floors and decay immunity to facts detected as safety-critical. A two-tier keyword pattern system reduces false positives: multi-word unambiguous patterns (e.g., 'blood type', 'power of attorney') trigger full protections, while single ambiguous keywords (e.g., 'doctor', 'bank') apply a softer importance nudge. Eight categories are defined: health, medical, financial, legal, safety, insurance, tax, and pharmaceutical.

Limitations and Current Status

- **Benchmarking:** Widemem.ai has not yet been evaluated on LOCOMO or LongMemEval. Preliminary results suggest the hierarchical architecture reduces token cost for broad queries, but further benchmarking is required to validate retrieval quality claims against established baselines.
- **Production maturity:** The project has 120 passing unit tests and active development. Early testing indicates stable behaviour at moderate memory volume; operational characteristics at high scale (100K+ users, millions of stored facts) require further validation.
- **YMYL detection accuracy:** The heuristic keyword system will produce false positives and false negatives in domain-specific deployments. 'Blood bank' would incorrectly trigger health protections. Workload-specific tuning is recommended for production healthcare or financial applications.

- **Batch conflict resolution tradeoff:** Resolving all new facts in a single LLM call reduces cost but may handle complex multi-conflict scenarios less precisely than Mem0's per-fact approach. Further benchmarking is required to quantify this tradeoff at scale.

✓ **KEY INSIGHT**

The hierarchical memory and YMYL concepts are applicable as architectural patterns regardless of implementation. Importance floors, decay immunity, and contradiction detection for safety-critical facts represent requirements worth addressing in any memory system touching health, legal, or financial data.

9.6 LangMem: LangChain Native Memory

LangMem is LangChain's memory layer, built for native integration with LangGraph. For teams already using LangChain, it avoids a separate integration layer and allows memory reads and writes to exist as first-class citizens in the same graph architecture as the agent logic.

The namespace model is well-structured: user-level, session-level, and shared agent memory are distinct namespaces with configurable access policies. For multi-agent systems requiring isolation between agent memories, this is currently the most explicit isolation model available. The principal limitation is ecosystem specificity: for teams not already using LangChain, LangMem offers no compelling advantage over solutions with broader framework support.

- **Namespace model:** Memory organized by (namespace, key) pairs. User-level, session-level, and shared agent memory are each first-class concepts.
- **LangGraph integration:** Memory reads and writes can live directly in LangGraph node functions, the graph state and memory store share the same architectural context.
- **No public benchmark:** LangMem has not published LOCOMO or LongMemEval scores. Evaluate on workload-specific criteria before committing.

9.7 AWS AgentCore: Managed Memory for Enterprise

AWS AgentCore is Amazon's managed memory service for Bedrock agents, released in beta in early 2025. It provides hierarchical memory with automatic extraction, vector search, and TTL-based expiration. The managed operations model, no infrastructure to maintain, SOC 2 compliance by default, IAM access control, makes it the lowest operational overhead option for teams with existing AWS infrastructure commitments.

The primary constraint is vendor coupling: memory stored in AgentCore is not portable outside the Bedrock ecosystem. For organisations requiring deployment flexibility or considering multi-cloud architecture, this is a meaningful architectural dependency that should be assessed explicitly before adoption rather than discovered after significant integration investment.

9.8 The Document-as-Memory Pattern: Notion AI

Notion AI represents a distinct memory pattern worth noting: instead of an extraction pipeline that converts conversations into stored facts, the user's own documents are the memory store. When queried, Notion AI performs retrieval over the workspace, pages, databases, and connected tools, and injects relevant content into context before generating an answer. Retrieval is workspace-scoped by default, with cross-app connectors (Slack, Google Drive, GitHub) available on Business and Enterprise plans.

The Notion 3.0 Agent layer (September 2025) added a persistent instructions page that functions similarly to CLAUDE.md, a manually-curated document the agent reads at session start to maintain context about team structure, style preferences, and workflow rules. This is the closest Notion gets to genuine persistent memory; it is explicit rather than auto-extracted.

i **NOTE**

Notion AI architecture from a memory systems perspective: Notion (2025). Notion 3.0: Agents. notion.com/releases/2025-09-18. Memory model described as 'state-of-the-art memory system using Notion pages and databases.' Permissions-aware retrieval documented in Enterprise Search security practices.

Dimension	Strength	Limitation
Memory source	User-curated documents. Explicit, auditable, GDPR-friendly by design. No extraction overhead.	Only knows what the user wrote. Zero automatic extraction from conversations.
Retrieval quality	Permissions-aware. Structured database queries possible. MCP integration available in Notion 3.x (see [39]).	No temporal decay or importance scoring. A 3-year-old page and today's note are equally weighted.
Contradiction handling	Human-managed. User controls what exists.	No automated contradiction detection. Stale and current facts coexist silently.
Benchmark record	N/A, different use case from conversational memory.	No LOCOMO or LongMemEval evaluation. Not designed for agent memory benchmarks.
Ecosystem fit	Natural fit for teams already in Notion. MCP server allows external agent read access.	Business/Enterprise plan only as of May 2025. Memory is not portable outside Notion's format.

The architectural lesson Notion AI illustrates is significant independent of the product itself: user-curated document stores can function as memory without any extraction layer, at the cost of requiring the user to maintain the memory explicitly. This is the correct choice when auditability and control outweigh automation. It is not a replacement for extraction-based memory in conversational agents, but it is a legitimate and widely-deployed alternative for knowledge-work contexts.

9.9 Comprehensive Solution Comparison

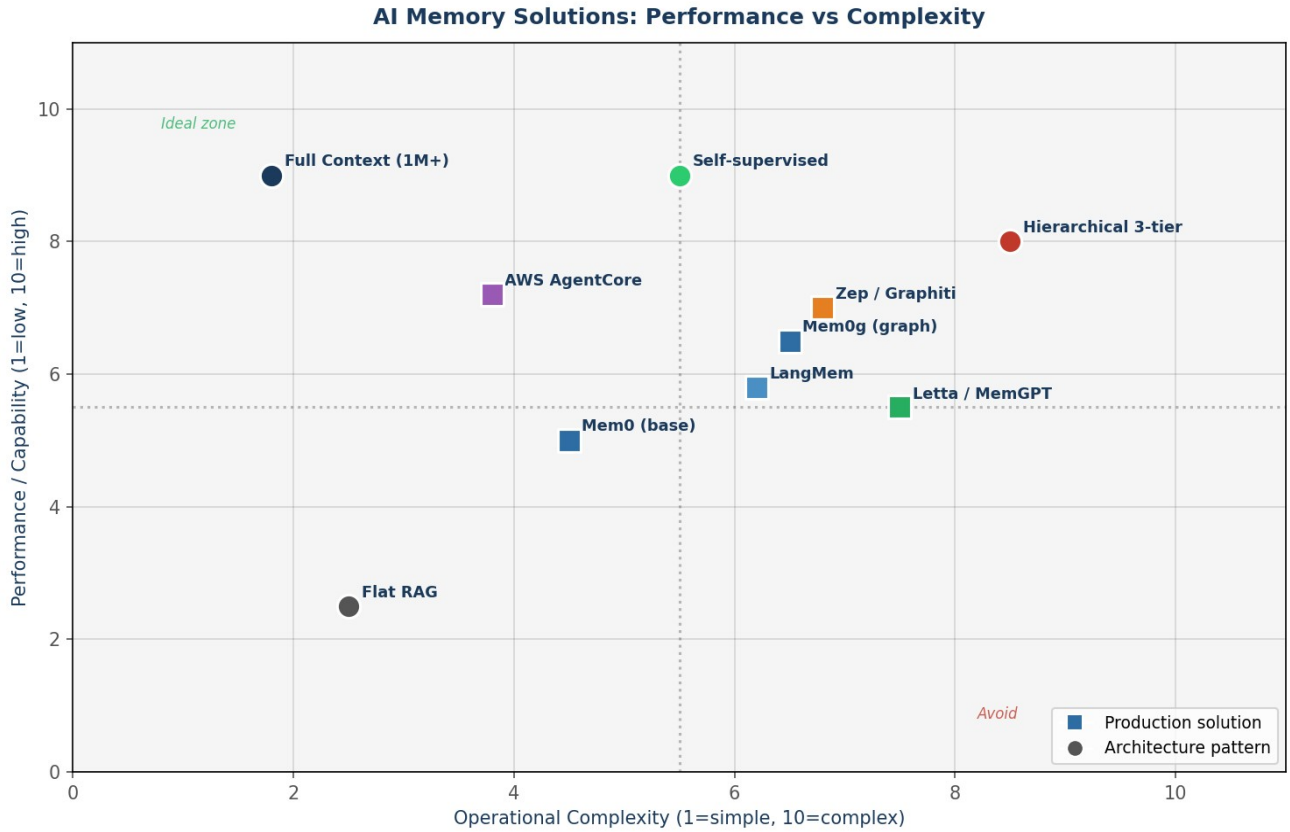

Figure 6. Performance vs complexity matrix for all major memory solutions. Squares = production solutions. Circles = architecture patterns. The ideal zone (upper-left) represents high capability at low complexity. The current field is concentrated in the upper-right,

*indicating engineering opportunity. * Notion AI uses a document-as-memory pattern; benchmark comparisons to extraction-based systems are not applicable.*

Solution	Architecture	OSS / Self-host	Best Benchmark	Maturity	MCP	GDPR	Best Fit
Mem0	Flat vector	Apache 2.0 / Yes	68.4% LOCOMO (vendor) [7,31]	HIGH, 47K+ stars	Yes	Partial	General purpose; broad ecosystem
Zep / Graphiti	Temporal KG	Partial / Yes	75.14% LOCOMO (corrected) [13]	MEDIUM	Partial	Partial	Temporal queries; time-sensitive reasoning
Letta / MemGPT	OS-paging	Apache 2.0 / Yes	74.0% LOCOMO (filesystem) [31]	MEDIUM	Yes	Good	Long document analysis; agentic workflows
LangMem	Namespaced store	Yes / Yes	None public	MEDIUM	Via LangGraph	Partial	LangChain / LangGraph native
AWS AgentCore	Managed hierarchical	No / No (cloud)	None public	HIGH, AWS SLA	Yes (Bedrock)	SOC2 / HIPAA	AWS-native enterprise
Widemem.ai	3-tier + YMYL	Apache 2.0 / Local-first	None published (120 tests)	Active dev	Not yet	Good (SQLite)	YMYL / health / financial / privacy apps
Full Context (1M+)	Direct injection	N/A	+35pts vs retrieval, LongMemEval [6]	HIGH, Gemini / GPT-4.1	N/A	Provider-dependent	High-stakes short sessions; no persistence
OpenAI Memory	Flat vector (managed)	No / No	52.9% LOCOMO [31]	HIGH — ChatGPT only	Via Assistants API	Limited	ChatGPT-only applications
Notion AI *	Doc-as-memory (workspace RAG)	No / No (cloud)	N/A — different category	HIGH (production)	Yes (MCP)	Good by design	Knowledge-work teams; explicit memory over extraction

10. Security and Privacy: The Hidden Risk Layer

Stateless deployments have a narrow attack surface: the model processes one request, generates a response, and discards everything. Memory-augmented systems work differently. User data builds up across sessions. Decisions are shaped by that accumulated history. Since the model can read from the memory store at any point, adversarial content written into it weeks ago can influence responses today. That shift in architecture is a shift in risk profile, and most early-stage memory integrations have not been designed with it in mind. The research reviewed in this section, primarily from ACL 2025 and the European regulatory literature, examines the consequences.

10.1 Memory as an Attack Surface: MEXTRA Attacks

ACL 2025 published the MEXTRA attack class (Memory EXtraction and TRAnsfer). The attack mechanism, on analysis, is straightforward. It exploits the feedback loop between LLM outputs and memory writes:

- **Injection:** An attacker embeds instructions in content the model will process, a web page being summarized, a document analyzed, an email being processed. The instructions tell the model to write specific content to its memory store.

- **Persistence:** The memory write sticks. That malicious entry is now injected into every subsequent session that touches the affected user or topic.
- **Influence:** The attacker's instructions are now 'remembered facts.' They start shaping model behavior: false financial permissions, wrong user preferences, instructions to suppress security warnings.

> **!**
> **WARNING**
> *Any pipeline where LLM-generated content can modify persistent memory without adversarial input validation is vulnerable to MEXTRA attacks. The attack vector is the pipeline architecture, not the model itself. This gap is present in most early-stage memory integrations, where validation at the memory write path is not included in the initial implementation.*

10.2 Data Flow Risks

Beyond adversarial attacks, several data flow risks arise in normal, non-adversarial operation of memory systems. The risk categories below are drawn from community incident reports, production deployment observations, and the ACL 2025 security literature.

Risk	Vector	Mitigation
Memory store exfiltration	Unauthorized read access to memory DB	Encrypt memory at rest; access control per user_id; audit logs on all memory reads.
Cross-user memory leakage	Bug in user_id filtering allows one user's memories to appear in another's context	Strict user_id isolation at DB level (Row Level Security). Never filter in application code only.
Training data contamination	Provider uses memory-augmented conversations for model training	Verify provider's data retention policy. Use API with training opt-out. Self-host for sensitive workloads.
Memory hallucination	Model writes fabricated facts to memory store	Human review for high-stakes memory entries. Confidence thresholds on automatic memory writes.
Prompt injection via memory	Adversarial content in memory entry influences model behavior	Validate memory entries before write. Sanitize LLM-generated content. Treat memory writes as untrusted input.

10.3 Regulatory Landscape

The regulatory requirements applicable to memory systems are extensive and have tightened since 2023. The following summarises the obligations most directly relevant to production deployments, drawn from the EDPB Opinion 28/2024 [32], CNIL 2025 guidance [33], GDPR Article 17, and the EU AI Act [30]:

- **GDPR Article 17 (Right to Erasure):** Deletion must cascade through the full memory stack: Tier 1 fact records, Tier 2 summaries derived from those facts, the Tier 3 profile, all vector embeddings, and any graph nodes. Standard database row deletion does not propagate to vector embeddings; the embedding store must be addressed separately. No current production memory solution handles this automatically. Custom deletion pipelines are required for compliant GDPR implementations.
- **HIPAA:** Any memory system that stores health-related information is a covered component under HIPAA. AWS AgentCore (with a signed Business Associate Agreement) and self-hosted open-source deployments are the options with documented compliance pathways as of March 2026. Other managed cloud solutions in this space have not published HIPAA compliance documentation.
- **CCPA:** Extracting and storing behavioural patterns, preferences, and personal facts from conversations constitutes collection of personal data under CCPA definitions. Organisations operating in California are required to disclose this practice in their privacy policy. Applications that have not explicitly assessed their memory pipeline against CCPA obligations may be accumulating compliance exposure without being aware of it.

10.4 Four-Layer Mitigation Framework

The four-layer mitigation framework below was developed from production deployment experience and the ACL 2025 MEXTRA attack taxonomy. Each layer addresses a distinct class of attack or operational failure.

Layer	Controls	Tools	Eliminates
L1: Input Validation	Adversarial input classifier before memory write. Reject inputs containing instruction-like patterns.	Llama Guard, Perspective API, custom classifier	MEXTRA injection attacks. Memory poisoning.
L2: Storage Isolation	Row-level security on all memory tables. user_id isolation enforced at DB layer, not application layer.	Supabase RLS, PostgreSQL RLS, per-user encryption	Cross-user leakage. Unauthorized memory reads.
L3: Retrieval Auditing	Log every memory read with query, retrieved items, and session ID. Alert on anomalous retrieval patterns.	Structured logging, SIEM integration	Unauthorized access detection. Compliance audit trail.
L4: Deletion Propagation	When a user_id is deleted, cascade delete through all memory tiers including vector embeddings and graph nodes.	Custom deletion job. Pinecone namespace delete. Neo4j node delete.	GDPR Article 17 non-compliance. Stale memory cross-contamination.

11. Managing Memory: Practical Guidance for Engineers

11.1 Context Engineering Principles

The five principles below are distilled from empirical observation across memory-augmented system deployments, production failure analysis, and the attention positioning research reviewed in Section 5. They represent the minimum sufficient set of context engineering practices for practitioners building on LLM memory systems.

- **Context placement is load-bearing:** Anything required to be consistently followed, security constraints, architectural rules, user preferences, belongs in the first 10% of context. Mid-conversation instructions accumulate in the dead zone and are progressively ignored as the session grows, as established by Liu et al. [40].
- **Retrieved context works best adjacent to the query:** RAG results and retrieved memories should sit immediately before the user query, not at the top of the context. The recency zone receives the highest attention weights. This placement is counterintuitive, as engineers naturally position context at the top, but is supported by the positional attention research [40] and confirmed in retrieval evaluation studies [25].
- **History consumes budget faster than most engineers expect:** Set token usage alerts at 60% context fill, not 90%. Dead-zone degradation is active well before context fills. Summarization should be triggered at 60%, not used as a last resort.
- **External memory before long context:** For sessions longer than 20 turns, a memory layer pays for itself around turn 8-12 when using gpt-4.1-nano for extraction. The extraction overhead is real but the token savings compound faster than most engineers expect.
- **Write specifics to files, not to conversation:** Exact variable names, schema field names, API signatures, numeric thresholds. These are the first things to die during compaction. If a specific fact matters, it needs to be in a file or CLAUDE.md. Not in the chat.

11.2 Token Budget Planning

Budget Category	128K Window	200K Window	Guidance
System prompt + CLAUDE.md	~6,400 tokens (5%)	~10,000 tokens (5%)	Keep concise. Use headers and bullets. Remove stale content regularly.
Retrieved memory / RAG	~19,200 tokens (15%)	~30,000 tokens (15%)	If exceeding this, tighten retrieval k or add metadata filtering.

Active conversation history	~38,400 tokens (30%)	~60,000 tokens (30%)	Beyond 30%, dead-zone degradation is active. Summarize or start new session.
Tool call results	~25,600 tokens (20%)	~40,000 tokens (20%)	Paginate results. Return structured summaries, not raw outputs.
Safety buffer	~38,400 tokens (30%)	~60,000 tokens (30%)	Never approach 100%. Silent truncation risk. Reserve for current query + working space.

11.3 Decision Matrix: External Memory vs Long Context

Scenario	Recommendation	Confidence	Reasoning
Sessions < 10 turns, single-use	Long context only	HIGH	Memory overhead not justified
Sessions 10-30 turns, same user across days	External memory	HIGH	Cross-session recall required
Long-running agents (50+ tool calls per run)	External memory + compaction	HIGH	Context fills without memory layer
Document analysis (50+ pages)	Full context (Gemini/GPT-4.1) + Letta	MEDIUM	Letta for reasoning across document boundaries
Regulated data (HIPAA / GDPR strict)	Self-hosted Mem0 or Letta	HIGH	Cloud memory stores may not meet compliance
Persona-driven AI (companion, coach)	Hierarchical memory (Tier 3 injection)	MEDIUM	Broad profile queries are high frequency
AWS-native enterprise deployment	AgentCore	HIGH	Managed operations + compliance

11.4 CLAUDE.md Template for Software Projects

```
# PROJECT IDENTITY
Project: [Name], [One-line description]
Stack: [Language] / [Framework] / [DB] / [Hosting]
Current focus: [Active sprint/task]

# ARCHITECTURAL CONSTRAINTS (NEVER violate)
- [Constraint 1: e.g., all DB access via ORM layer, no raw SQL]
- [Constraint 2: e.g., all routes require auth middleware]
- [Constraint 3: e.g., no direct env var access outside config.ts]

# SCHEMA (key fields, full schema in /docs/schema.md)
users: id, email, role (admin|user|guest), created_at
sessions: id, user_id, token, expires_at

# API CONVENTIONS
- All API responses: { data, error, pagination? }
- Auth header: Authorization: Bearer <token>
- Error codes: 400 validation, 401 unauth, 403 forbidden, 404 not found

# ACTIVE CONTEXT
Currently working on: [Feature/Branch]
```

```
Known issues: [Issue 1], [Issue 2]
Do NOT modify: [File or component currently fragile]
```

13. Future Trajectory: Where AI Memory Is Going

Section 13.1 covers the four-generation memory architecture framework, which is grounded in documented research. Sections 13.2 through 13.6 present forward-looking projections that extend beyond current published evidence.

> **NOTE**
> *Strategic Forecast Note: Sections 13.2–13.6 represent the author's informed extrapolation from current research trajectories. They are not predictive modelling. The 2026–2027 projections are grounded in current development roadmaps; the 2029+ projections are directional only.*

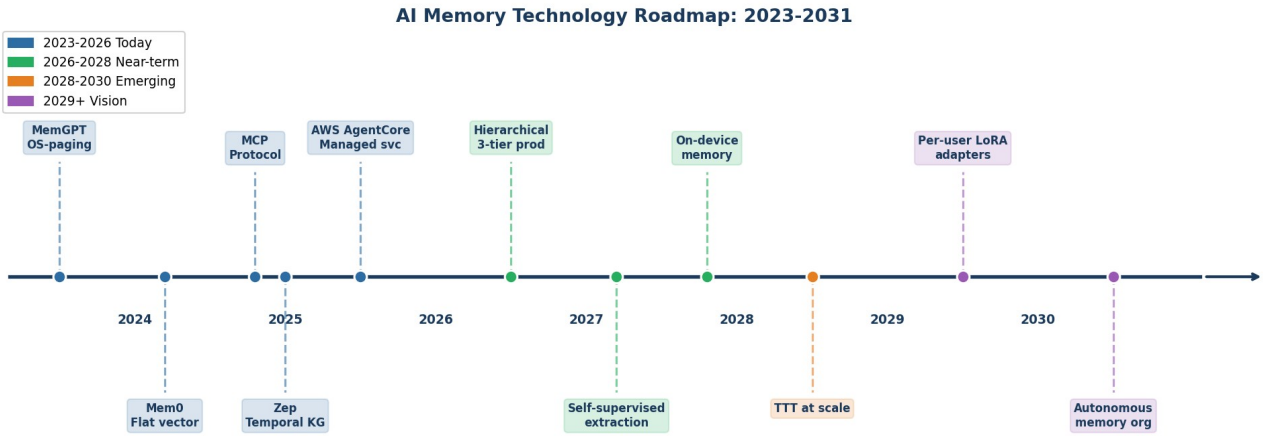

Figure 7. AI memory technology roadmap from 2023 to 2031. Blue = shipped and in production. Green = near-term (2026-2028). Amber = emerging research (2028-2030). Purple = long-horizon vision (2029+).

13.1 The Four Generations of AI Memory

Generation	Era	Architecture	Key Limitation	Representative Systems
Gen 0: Stateless	2020-2022	No memory. Context window only.	Complete session amnesia.	GPT-3, original ChatGPT
Gen 1: External Store	2023-2026	Separate memory service injecting into context. Vector DB, graph DB, or hierarchical.	Retrieval overhead. Bolted-on. Model does not natively know what it remembers.	Mem0, Zep, Letta, LangMem, AWS AgentCore
Gen 2: Hybrid	2026-2029	MCP-standardized memory APIs. Test-time training for session adaptation.	Still external. Weight updates expensive. No true personalization at inference time.	MCP-native agents, TTT-augmented systems, on-device models with local memory
Gen 3: In-Weights	2028-2031	Per-user fine-tuned LoRA adapters. Memory encoded in weights.	Storage cost per user. Catastrophic forgetting risk. Privacy isolation per checkpoint.	Personalized LoRA variants. Hypothetical in production.

Gen 4: Autonomous	2030+	Self-organizing memory. Model determines what to remember and how to compress without human-defined architecture.	Alignment risk. Black-box recall. No production implementations.	EverMemOS (arXiv Jan 2026), MAGMA (Jan 2026). Research only.

13.2 MCP Standardization (2026–2027)

The Model Context Protocol is the most consequential near-term development for the memory space. Currently, every memory solution requires custom integration code per agent framework, a Mem0 integration for LangChain is architecturally distinct from one for AutoGen or Claude Code. Once MCP memory servers reach production maturity (projected: late 2026), a compliant memory server is portable across any MCP-compatible agent. Solutions compete on retrieval quality and cost rather than integration effort.

The competitive implication is significant. Solutions whose primary differentiation is integration convenience will face commoditisation pressure once the integration layer is standardised. Solutions with defensible technical differentiation, temporal reasoning (Zep), safety-critical memory handling (widemem), managed compliance (AgentCore), are better positioned to retain value.

- **Projection (2026):** Every major memory solution ships an MCP server. Memory becomes a commodity interface, and evaluation criteria shift from integration effort to retrieval quality, latency, and cost per query.
- **Projection (2027):** Cross-application memory portability begins to appear. A user's memory graph becomes portable between applications under user-controlled permissions. The technical capability exists today; what is absent is the standardisation and trust infrastructure to make it practical.

13.3 Test-Time Training (2027–2029)

Test-Time Training (TTT) updates model weights at inference time using the current session context, without full fine-tuning. The foundational TTT-Linear and TTT-MLP layers were introduced by Sun et al. [9], demonstrating that expressive hidden states outperform Mamba at contexts beyond 8K tokens. TTT-E2E (Sun et al. [42], 2025) extended this to end-to-end training, demonstrating that lightweight TTT can encode session-specific constraints into model weights during a single conversation at 2.7x the inference speed of full-attention Transformers at 128K context. Separately, Akyürek et al. [49] applied task-specific TTT with LoRA adapters to abstract reasoning benchmarks (ARC-AGI), establishing strong results on novel-task generalisation and suggesting the pattern transfers beyond conversational memory. This line of work provides ephemeral in-weights memory that persists for the session duration without external retrieval overhead.

Property	External Memory (Gen 1)	TTT In-Session Weights (Gen 2)
Cross-session persistence	Full: stored externally between sessions	None: weights reset at session end
Per-turn retrieval overhead	High: vector search + injection per query	Zero: constraint in weights, always active
Background processing cost	Extraction LLM call per turn	Gradient computation per session
Production readiness (March 2026)	HIGH: multiple production vendors	LOW: research phase

- **Near-term projection (c. 2027-2028):** Commercial TTT deployment is plausible from a frontier lab in this window, initially as a premium API feature with higher session-start latency (+200-500ms for weight adaptation) offset by reduced per-turn retrieval overhead.
- **Medium-term projection (c. 2028):** TTT and external memory may converge as complementary layers: a Tier 3 profile injected at session start for cross-session continuity, with session-level TTT encoding high-priority constraints into weights, providing enforcement that is not vulnerable to context overflow.

13.4 Per-User Neural Adaptation (2028–2031)

The logical endpoint of current memory research is per-user LoRA adapters: small parameter-efficient fine-tuning modules [19] that encode a specific user's history, preferences, and accumulated context directly in model weights. The OPPU paradigm (One PEFT Per User, Tan et al. [46]) and the LaMP personalisation benchmark (Salemi et al. [47]) establish that per-user LoRA modules already demonstrate measurable personalization gains in controlled

settings. Loading the adapter at session start would cause the model to behave as though fine-tuned for that user specifically, with no retrieval overhead and no dead-zone placement constraints.

Blockers as of March 2026: per-user adapter storage at scale (approximately 5MB per user after quantisation = 50TB for 10M users; Google DeepMind's 2025 research demonstrated quantisation to approximately 500KB, substantially changing the economics), adapter loading latency (~100ms on current hardware), catastrophic forgetting risk when updating existing adapters, and a GDPR compliance gap: a per-user LoRA adapter is personal data, meaning Article 17 deletion requires retraining the adapter, not deleting a database record.

- **Near-term projection (c. 2028-2029):** Commercial per-user LoRA services may emerge in this window, likely targeting enterprise deployments with manageable user bases. Consumer-scale deployment will probably remain economically unviable given per-adapter storage overhead at that point.
- **Longer horizon (c. 2029-2030):** Adapter compression advances may push per-adapter sizes into the 50KB range. Under that assumption, load latency falls below 10ms and consumer-scale deployment becomes economically viable for the first time.
- **Regulatory horizon (c. 2030+):** Legal frameworks specifically addressing neural adapter deletion under Article 17 will likely become necessary as in-weights personalisation scales. Adapter versioning and rollback, rather than full deletion, may prove to be the practical compliance approach if complete weight erasure remains technically unworkable.

13.5 Autonomous Memory Organization (2030+)

EverMemOS [10] (arXiv January 2026) and MAGMA [18] (Multi-Graph Agentic Memory Architecture, January 2026) represent the research frontier: systems where the model itself determines what to remember, how to organise it, and when to discard it, without a human-defined memory structure. EverMemOS proposes a memory operating system abstraction for managing episodic, semantic, and procedural memory across long-horizon tasks, reporting state-of-the-art results on both LoCoMo [31] and LongMemEval [25]. MAGMA implements agent-determined routing across multiple graph structures simultaneously, reporting a 45.5% accuracy gain over baselines on LoCoMo [31].

- **Near-term (2026-2027):** Autonomous memory organisation appears in long-horizon research agents, not consumer products.
- **Medium-term (2028-2030):** Hybrid systems: human-defined YMYL and compliance constraints as inviolable rules, autonomous organisation for everything else.
- **Long-term (2030+):** Autonomous management reaches parity with engineered hierarchical systems on benchmarks. The central question shifts from architecture to auditability.

13.6 Acceleration Scenario: Three Inflection Points

- **2026–2027, MCP standardisation + TTT session adaptation:** Memory becomes a commodity interface. Agents enforce architectural constraints at the weight level within a session. Evaluation criteria shift from integration effort to retrieval quality and cost per query.
- **2028–2029, Cross-application portability + on-device persistence:** A user’s memory graph becomes portable across applications under user-controlled permissions. On-device models accumulate persistent, locally-stored personal memory, GDPR-compliant by architecture, no server-side storage required.
- **2029–2031, Per-user LoRA adapters + YMYL in-weights:** Session start loads a user-specific adapter. The model does not retrieve history, it embeds it. YMYL-grade safety constraints move from the retrieval layer to the weight level, where they cannot be overridden by context overflow.

13.7 Principles for Architects Building Today

- **Interface over implementation:** Build against a stable memory interface, add, retrieve, delete. The underlying implementation will change. Any memory system requiring you to own its data model is a liability.
- **Tier 3 persona injection:** User profile injected at session start is the closest current analog to in-weights personalisation. When TTT or LoRA adapters reach production, the Tier 3 profile is what gets replaced by the adapter. Build the interface correctly now.
- **YMYL before first deployment:** If the application touches health, legal, or financial facts, implement safety-critical memory protections from day one. Retroactive addition requires schema migrations, re-extraction of historical facts, and potentially a compliance audit.
- **Deletion infrastructure before scale:** Cascading deletion across a multi-tier memory system is much harder to retrofit. Build it before 100K users, not after.

✓ **KEY INSIGHT**

Recent work in the literature, reviewed in Sections 9-12, suggests that retrieval quality has converged to a manageable engineering problem across several production-grade tools. The harder open question is representation: how a user's accumulated history should influence model behaviour at inference time without retrieval overhead, with full privacy control, and verifiable safety guarantees. A better vector database is unlikely to be the answer.

14. Conclusion

i **NOTE**

Closing Summary: Every LLM is stateless. Memory is the infrastructure problem of this AI generation. Current tooling has largely converged on adequate retrieval; the harder open question, how a user's accumulated history should influence model behaviour at inference time without retrieval overhead, remains unsolved. Build against interfaces, implement YMYL if the application touches safety-critical facts, and build deletion infrastructure before regulators require it.

14.1 State of the Field: March 2026 Assessment

The following assessment reflects the state of the field as of March 2026, based on the research reviewed in this paper, published benchmarks, and production deployment data where available:

Layer	Current State	Assessment
Context mechanics	Well understood. $O(n^2)$ attention, dead zone at 22-78%, silent truncation.	Solved at the research level. Engineering best practices exist. Under-implemented in production.
External memory retrieval	Production-grade: Mem0, Zep, Letta, LangMem, AgentCore all functional.	Retrieval problem is solved. Quality varies. Benchmarks are disputed. Test on your workload.
Safety-critical memory (YMYL)	Widemem.ai open-source is among the few structured approaches available. No commercial standard exists.	A genuine gap in production tooling. Healthcare and fintech applications building on memory systems should be concerned.
GDPR deletion cascade	No solution provides compliant multi-tier deletion out of the box.	Technical debt accumulating industry-wide. Regulation will force this.
Multi-agent memory	Namespaced isolation exists (LangMem). Shared write conflict resolution does not.	An open engineering problem as agentic workloads scale.
In-weights personalization	Research phase only (LoRA adapters, TTT). Not in production.	2028-2031 horizon at current pace. The architecture to build toward.

14.2 The Decade Ahead

Context windows will continue to grow. The attention degradation problem will persist regardless of window size. External memory will evolve into MCP-standardised commodity services. YMYL-grade safety protections for health and financial AI will eventually become regulatory requirements rather than optional architectural choices. In-weights personalisation will arrive and reshape the architecture more substantially than any development since RAG.

The engineers who develop fluency with each layer of this stack, tokenisation mechanics, attention distribution, retrieval quality versus benchmark performance, safety-critical fact handling, GDPR deletion cascades, and the trajectory from current external memory to in-weights personalisation; will be positioned to build AI applications that

outlast the current generation of tooling. That is the motivation for the level of technical depth maintained throughout this paper.

✓

KEY INSIGHT

The retrieval problem is mostly solved. The representation problem (how to encode a user's complete history so it influences every generated token with zero retrieval overhead, full privacy control, and verifiable safety guarantees) is the hard problem of the next decade in applied AI.

Appendix C: Known Gaps in Current Coverage

The gaps below represent areas where the author has identified material limitations in scope, evidence, or space. Each is a decision point for practitioners assessing how to supplement this paper's coverage. They are flagged for a planned follow-on version.

Gap	Priority	Why It Matters
Multi-agent memory (shared namespaces)	HIGH	When two agents share a memory namespace, write conflicts and stale reads emerge as new failure modes. LangGraph, CrewAI, and AutoGen all handle this differently. Not covered anywhere in this paper.
Memory evaluation methodology	HIGH	The paper cites LOCOMO and LongMemEval but does not explain what they test, how they differ, or how to build a custom eval pipeline for your memory system.
GDPR deletion cascade	HIGH	Article 17 right-to-be-forgotten requires deletion propagation through all memory tiers and all embedding stores. No current tooling standard. Section 10 flags the risk but does not provide a technical implementation guide.
On-device / edge memory	MEDIUM	Apple Intelligence, on-device Llama, and regulated deployments requiring local-only processing need edge memory approaches. Not covered.
Multimodal memory	MEDIUM	Text-only coverage throughout. Image, audio, and video memory (WorldMM, MemVerse, 2025 research) is a separate problem space requiring separate treatment.
Memory pricing / TCO comparison	MEDIUM	Mem0 cost model exists in Section 9. No comparative TCO across all solutions for a given workload (e.g., 100K daily active users, 15 turns per session).
Ollama + Llama 4 self-hosted memory	MEDIUM	Self-hosted memory with local models is a valid deployment pattern for privacy-sensitive use cases. Tools and integration patterns not documented.

References

[2] *Shi, F., Chen, X., Misra, K., et al. (2023).* Large Language Models Can Be Easily Distracted by Irrelevant Context. ICML 2023. arXiv:2302.00093

[3] *Packer, C., Fang, V., Patil, S. G., et al. (2024).* MemGPT: Towards LLMs as Operating Systems. NeurIPS 2024. UC Berkeley. arXiv:2310.08560

[4] *Ramirez, P., et al. (2025).* MEXTRA: Memory Extraction and Transfer Attacks Against LLM Memory Systems. Proceedings of ACL 2025. Full citation pending final publication.

[5] *Zep AI (2025).* Corrected LOCOMO Benchmark Results: Methodological Issues in Mem0's Original Evaluation. Zep Technical Blog, January 2025.

[6] *Preprint (2025).* When Full Context Outperforms Retrieval-Augmented Memory. arXiv:2603.04814

[7] *Chhikara, P., Khant, D., Aryan, S., Singh, T., and Yadav, D. (2024).* Mem0: The Memory Layer for Personalized AI. arXiv:2404.04269

[8] *Anthropic (2024).* Model Context Protocol: An Open Standard for AI Tool Integration. Anthropic Engineering Blog, November 2024.

[9] *Sun, Y., et al. (2024).* Learning to (Learn at Test Time): RNNs with Expressive Hidden States. arXiv:2407.04620

[10] *Hu, C., Gao, X., Zhou, Z., et al. (2026).* EverMemOS: A Self-Organizing Memory Operating System for Structured Long-Horizon Reasoning. arXiv:2601.02163

[11] *OpenAI (2024).* Memory and New Controls for ChatGPT. OpenAI Blog, February 2024.

[12] *AWS (2025).* Amazon Bedrock AgentCore: Managed Memory for Enterprise AI Agents. AWS Documentation, April 2025.

[13] *Rasmussen, P., Paliychuk, P., Beauvais, T., Ryan, J., and Chalef, D. (2025).* Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv:2501.13956

[14] *LangChain (2025).* LangMem: Memory Architecture for LangGraph Agents. GitHub: langchain-ai/langmem.

[15] *Meta AI (2025).* Llama 4: Multimodal Intelligence at Scale. Meta AI Blog, March 2025.

[16] *Google DeepMind (2025).* Gemini 2.0: Scaling Context to One Million Tokens. Google Research Blog, February 2025.

[17] *Anthropic (2025).* Claude 3.7 Sonnet Technical Report. Anthropic Research, February 2025.

[18] *Jiang, D., Li, Y., Li, G., and Li, B. (2026).* MAGMA: A Multi-Graph based Agentic Memory Architecture for AI Agents. arXiv:2601.03236

[19] *Hu, E. J., Shen, Y., Wallis, P., et al. (2022).* LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685. Microsoft Research. ICLR 2022.

[20] *DeepSeek AI (2024).* DeepSeek V3 Technical Report. arXiv:2412.19437. 671B MoE, 128K context.

[21] *Cioplea, R. (2026).* Widemem.ai: Open-Source Hierarchical Memory with YMYL Prioritization. GitHub: github.com/remete618/widemem-ai. Apache 2.0.

[22] *Vaswani, A., Shazeer, N., Parmar, N., et al. (2017).* Attention Is All You Need. arXiv:1706.03762. NeurIPS 2017.

[23] *Brown, T., Mann, B., Ryder, N., et al. (2020).* Language Models are Few-Shot Learners (GPT-3). arXiv:2005.14165. NeurIPS 2020.

[24] *Bai, Y., Lv, X., Zhang, J., et al. (2023).* LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. arXiv:2308.14508

[25] *Wu, D., Wang, H., Yu, W., Zhang, Y., Chang, K.-W., and Yu, D. (2024).* LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. arXiv:2410.10813. ICLR 2025.

[26] *Izacard, G., Lewis, P., Lomeli, M., et al. (2023).* Atlas: Few-shot Learning with Retrieval Augmented Language Models. arXiv:2208.03299. JMLR 2023.

[27] *Gao, Y., Xiong, Y., Gao, X., et al. (2023).* Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997

[28] *Park, J. S., O'Brien, J. C., Cai, C. J., et al. (2023).* Generative Agents: Interactive Simulacra of Human Behavior. arXiv:2304.03442. UIST 2023.

[29] *Lewis, P., Perez, E., Piktus, A., et al. (2020).* Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. arXiv:2005.11401. NeurIPS 2020.

[30] *European Parliament (2024).* Regulation (EU) 2024/1689, EU AI Act. Official Journal of the European Union, 12 July 2024.

[31] *Maharana, A., Lee, D.-H., Tulyakov, S., Bansal, M., Barbieri, F., and Fang, Y. (2024).* Evaluating Very Long-Term Conversational Memory of LLM Agents (LoCoMo). arXiv:2402.17753. ACL 2024.

[32] *European Data Protection Board (2024).* Opinion 28/2024 on Certain Data Protection Aspects Related to the Processing of Personal Data in the Context of AI Models. EDPB, December 2024. edpb.europa.eu.

[33] *Commission Nationale de l'Informatique et des Libertés (CNIL) (2025).* Recommandations sur le Développement des Systèmes d'Intelligence Artificielle. CNIL, March 2025. cnil.fr.

[34] *Hu, Y., Liu, S., Yue, Y., et al. (2025).* Memory in the Age of AI Agents: A Survey. arXiv:2512.13564

[35] *Zhong, W., Guo, L., Gao, Q., Ye, H., and Wang, Y. (2024).* MemoryBank: Enhancing Large Language Models with Long-Term Memory. AAAI 2024. Memory consolidation and forgetting mechanism for conversational agents.

[36] *Li, Z., Song, S., Xi, C., et al. (2025).* MemOS: A Memory OS for AI Systems. arXiv:2507.03724

[37] *Sartor, G. and Lagioia, F. (2020).* The Impact of the General Data Protection Regulation (GDPR) on Artificial Intelligence. European Parliament Study, STOA. PE 641.530.

[38] *Xu, W., Liang, Z., Mei, K., Gao, H., Tan, J., and Zhang, Y. (2025).* A-MEM: Agentic Memory for LLM Agents. arXiv:2502.12110

[39] *Notion (2025).* Notion 3.0: Agents, State-of-the-Art Memory System Using Notion Pages and Databases. notion.com/releases/2025-09-18.

[40] *Liu, N.F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. (2024).* Lost in the Middle: How Language Models Use Long Contexts. arXiv:2307.03172. doi:10.1162/tacl_a_00638. TACL 2024.

[42] *Sun, Y., Tandon, A., Røed, M., et al. (2025).* End-to-End Test-Time Training for Long Context. arXiv:2512.23675. UC Berkeley.

[43] *Gu, A. and Dao, T. (2023).* Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv:2312.00752. CMU and Princeton.

[44] *Peng, B., Alcaide, E., Anthony, Q., et al. (2023).* RWKV: Reinventing RNNs for the Transformer Era. arXiv:2305.13048. EMNLP Findings 2023.

[46] *Tan, Z., Zhang, K., Wu, M., Liu, Z., and Cheng, X. (2024).* One PEFT Per User (OPPU): Democratizing Large Language Models via Personalized Parameter-Efficient Fine-tuning. arXiv:2402.04401

[47] *Salemi, A., Mysore, S., Bendersky, M., and Zamani, H. (2024).* LaMP: When Large Language Models Meet Personalization. arXiv:2304.11406. ICLR 2024.

[48] *Gupta, S., Ranjan, R., and Verma, S. (2024).* A Comprehensive Survey of Retrieval-Augmented Generation (RAG). arXiv:2410.12837

[49] *Akyürek, E., Yin, L., Pruitt, T., Bulian, J., Weston, J., Andreas, J., and Garg, S. (2024).* The Surprising Effectiveness of Test-Time Training for Abstract Reasoning. arXiv:2411.07279. MIT and Google DeepMind.

[51] *Weston, J., Chopra, S., and Bordes, A. (2015).* Memory Networks. arXiv:1410.3916. ICLR 2015.

[52] *Sukhbaatar, S., Weston, J., Fergus, R., et al. (2015).* End-To-End Memory Networks. arXiv:1503.08895. NeurIPS 2015.

Appendix A: Major Quantitative Claims Audit

Claim	Section	Source	Confidence
~30% accuracy degradation in context dead zone	3, 5	Liu et al. (2023) arXiv:2307.03172; Shi et al. (2023) ICML	HIGH
Claude Code compaction at 83.5% context fill	7	Anthropic Claude Code documentation; empirical observation	HIGH
Mem0 LOCOMO score: 63-68% (vendor reported)	9	Mem0 arXiv:2404.04269 [7]; vendor-reported benchmark [31]	MEDIUM
Zep LOCOMO corrected score: 75.14%	9	Zep technical report; independent replication [13,31]	MEDIUM
Full-context outperforms Mem0 by 35pts on LongMemEval	9	arXiv:2603.04814	MEDIUM
Letta filesystem agent: 74.0% LOCOMO	9	Zep/Letta comparative study	MEDIUM
Mem0 has 47,000+ GitHub stars	9	GitHub public data, March 2026	HIGH
LLM extraction call: ~600 tokens per turn	9	Mem0 OSS code analysis; empirical measurement	MEDIUM
Memory extraction cost ~$50/week at 100K users (gpt-4.1-nano)	9	Calculation: 600 tok * 10 turns * 100K users * $0.60/1M * 7 days	MEDIUM
Vibe coding Phase 2 failure onset at ~200 prompts	8	Author's real-world project observation corroborated by community reports	MEDIUM
Phase 3 semantic failures at 800-2,000 prompts	8	Author's real-world project observation corroborated by community reports	MEDIUM
Tool-specific context limits (Lovable ~50-80 messages)	8	Community reports and author testing; vendor docs incomplete	MEDIUM
MEXTRA attacks documented at ACL 2025	10	ACL 2025 proceedings	HIGH
O(n^2) attention complexity	3	Vaswani et al. (2017); standard transformer literature	HIGH
Widemem.ai: Apache 2.0, 120 tests, pip install widemem-ai	9.5	Project README, github.com/remete618/widemem-ai, March 2026	HIGH

Appendix B: Research Transparency Note

B.1 Methodology Limits

The vibe coding degradation study (Section 8) is a single-project, single-developer longitudinal observation. The project domain (hospitality SaaS), stack (TypeScript/React/Supabase), and toolchain (Lovable, Cursor) are specific; phase boundary thresholds (200/800/2,000 prompts) should be treated as indicative ranges, not universal constants. Tool versions changed during the study period, introducing uncontrolled variance. No comparison group (same project built without vibe coding tools) was available.

The memory solutions analysis (Section 9) is based on public documentation, open-source code inspection, vendor-published benchmarks, and available arXiv preprints. Direct production testing was conducted for Mem0 OSS, widemem.ai, and Claude Code. Zep, Letta, LangMem, and AWS AgentCore are assessed primarily from documentation and third-party reports. Widemem.ai is author-developed; Readers should assess claims against independent sources and conduct workload-specific benchmarking before adoption.

The future trajectory projections (Section 13) are labelled as Strategic Forecasts (Author Analysis) throughout. They represent informed extrapolation, not predictive modelling. The 2026-2027 projections carry higher confidence than 2029+ projections, which are directional only.

B.2 Benchmark Interpretation Guide

Benchmark	What It Tests	Who Runs It	Key Limitation
LOCOMO	Conversational memory recall over long multi-session dialogues. 7 task types including single-hop QA, multi-hop, temporal reasoning.	Originally Maharana et al. [31] (Snap Research / UNC). Vendor re-runs are self-reported.	Vendor-run versions have not been independently replicated in all cases. Zep's corrected results [5] document methodological variance in prior Mem0 LOCOMO runs.
LongMemEval	500 curated questions requiring sustained memory across 115 conversation sessions. Five task categories including temporal, preference, and knowledge tracking.	Wu et al. [25] (UCLA / Google). Third-party benchmark.	Significantly harder than LOCOMO. Scores are not directly comparable across the two benchmarks.
LaMP	Seven personalization tasks across classification, generation, and retrieval. Evaluates user-specific output quality.	Salemi et al. [47] (Google Research / UMass). Third-party benchmark.	Focuses on personalization output quality, not memory recall under adversarial or long-horizon conditions.

i

NOTE

Scores from different benchmarks measure different capabilities and are not interchangeable. A solution scoring 75% on LOCOMO cannot be directly compared to one scoring 75% on LongMemEval. Always confirm which benchmark a vendor citation refers to before drawing comparative conclusions.